



Politehnica University of Timisoara



Web Programming

C3. Server-Side Scripting



Server-Side Scripting

- Alternative for “Network Application Programming” (CORBA, Socket, etc.)
- Code that runs on a Web server or implements the server (e.g., NodeJS)
- Two ways:
 - Embedded in HTML files (scripts, e.g., PHP)
 - External code: CGI, Java Servlets, .NET Web API
- Special case: JSP, ASP



PHP

- Designed for Web scripting
- Embedded within a HTML page
- PHP interpreter as Web server module
- PHP code generates HTML
- PHP code does not reach the client browser



PHP Applications

- Using PHP it is easily to build:
 - User authentication mechanism
 - Content management systems
 - Web based e-mail clients
 - Message list management
 - Web forums
 - PDF documents generated on the fly



PHP Syntax

- Escaping from HTML
 - A php code block starts with **<?php** and ends with **?>**
- Instruction separation
 - a semicolon at the end of each statement (except the last line of a PHP block)
- PHP supports C, C++ and Unix shell-style comments (**/******* ... *******/**, **//**, **#**)



Specific PHP Operators

- Error Control Operators
 - @
- Execution Operators
 - `` (ex. \$out = `ls -al`)
- Logical Operators
 - && and || or ! xor
- String Operators
 - concatenation .'



PHP Variables

- PHP is a loosely typed language
- No need for variable declaration
- All variables in PHP start with a **\$**
- Constants declaration (no **\$** before name)

```
define("PI", "3.14");
```

- Transparent type casting

```
$n = 1;  
$s = "1";  
$n1 = $n + $a;    // $n1=2 (integer)  
$n2 = $n1 + "10 little penguins"  
                // $n2=12 (integer)
```



PHP Casting issues

- Can generate ambiguities (sci notation of num)

```
<?php
    $passwd1 = md5("240610708");
    $passwd2 = md5("QNKCDZO");

    echo $passwd1." == ".$passwd2;
    echo "<br>";
    echo ($passwd1 == $passwd2 ? "true" : "false");
?>
```




PHP Predefined Variables

- `$GLOBALS` — References all variables available in global scope
- `$_SERVER` — Server and execution environment information
- `$_GET` — HTTP GET variables
- `$_POST` — HTTP POST variables
- `$_FILES` — HTTP File Upload variables
- `$_SESSION` — Session variables
- `$_ENV` — Environment variables
- `$_COOKIE` — HTTP Cookies



PHP Arrays

- Ordered map
- Associate values to keys
- A key: an integer or a string
- A value: any PHP type
- Multidimensional arrays
- Array creation: `array()`

```
$a = array( "p1" => "orange",  
           "p2" => "banana",  
           "p3" => "apple");  
print_r($a);
```



PHP Instructions

- if/else/elseif

```
if ($a > $b) {  
    echo "a is bigger than b";  
} elseif ($a == $b) {  
    echo "a is equal to b";  
} else {  
    echo "a is smaller than b";  
}
```

- switch/case/default/break

- C like syntax and semantics

- while / do while

- C like syntax and semantics



PHP Loops and array parsing

■ for

```
for ($i = 1; $i <= 10; $i++) {  
    echo $i;  
}
```

■ foreach

```
foreach (array_expression as $value)  
    statement  
foreach (array_expression as $key => $value)  
    statement
```

■ Loop control: break, continue



PHP Functions

```
function foo($arg_1, $arg_2, /* ..., */ $arg_n)
{
    echo "Example function.\n";
    return $retval;
}
```

- PHP does not support function overloading or redefinition (except for methods)
- Allow recursive definition
- Allow a variable number of arguments
- Arguments are passed by value except for arrays and objects



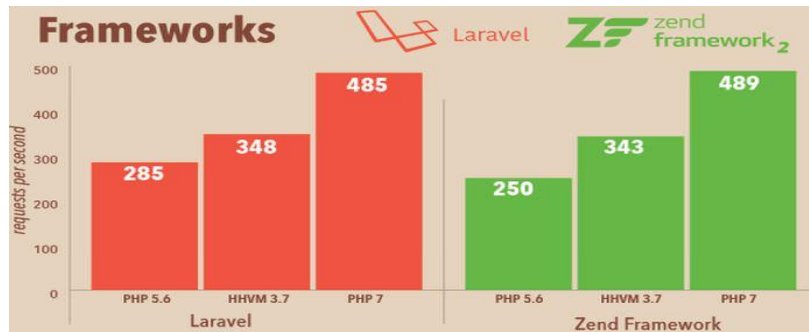
PHP Code Reuse

- **include**
 - Includes and evaluates the specified file
 - Will emit a warning if it cannot find the file
- **require**
 - Same with include
 - Will emit a fatal error if it cannot find the file
- To avoid unconditionally inclusion:
`eval('require filename;');`



PHP 7 (Dec. 2015)

- Performance improvement, memory optimization

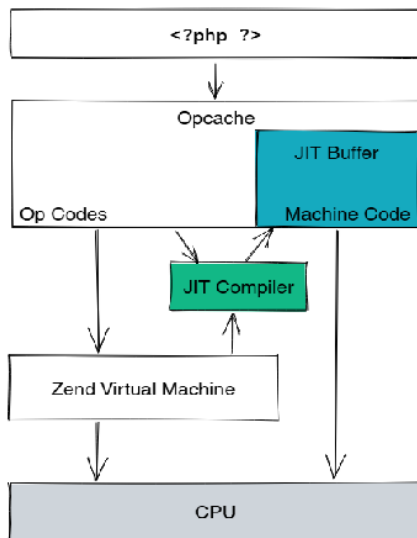


- Better than HipHop Virtual Machine (Facebook HHVM)
 - ZendEngine compiled intermediate OpCode
- New Engine Exceptions – fatal errors as exceptions
- Accurate Type Declarations (function return type)
- Anonymous Classes

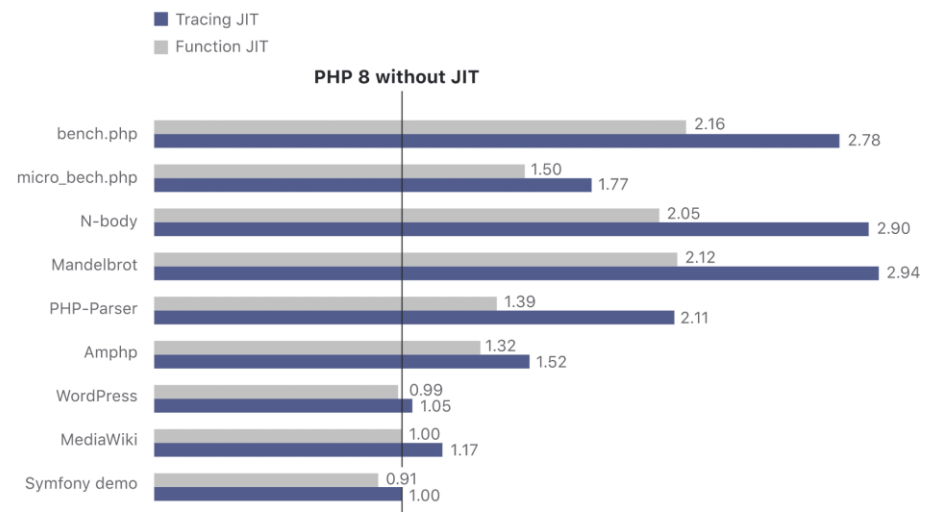


PHP 8.4 (Nov. 2024)

- JIT based on DynASM (Dynamic Assembler for code generation engines) to generate native machine code instead of OpCode



Relative JIT contribution to PHP 8 performance





PHP OOP

- PHP has a full object model starting with version 5
- Features:
 - Classes and Interfaces
 - *new* and object references
 - Inheritance through *extends*
 - Member Visibility: *public*, *protected*, *private*
 - *abstract* classes and methods
 - *final* class and methods



PHP. Methods overriding

- PHP allows methods overriding

```
class A {  
    function myName () {  
        return "A";  
    }  
}  
  
class B extends A {  
    function myName () {  
        return "B";  
    }  
}
```



PHP. Methods overloading

- No classic overloading. Alternatives:
_call(\$method, \$args) or via variable sized *args*

```
class A {  
    public function add() {  
        $sum = 0;  
        $args = func_get_args();  
        foreach ($args as $arg) $sum += $arg;  
        return $sum;  
    }  
}  
  
$a = new A();  
echo $a->add(1,2,3,5);
```



PHP 8 New Features

- Constructor Property Promotion

```
class Point {  
    public int $x;  
    public int $y;  
    public int $z;  
  
    public function __construct(  
        int $x = 0,  
        int $y = 0,  
        int $z = 0,  
    ) {  
        $this->x = $x;  
        $this->y = $y;  
        $this->z = $z;  
    }  
}
```

```
class Point {  
    public function __construct(  
        public int $x = 0,  
        public int $y = 0,  
        public int $z = 0,  
    ) {}  
}
```

- Incompatible Method Signatures
- error instead just a warning
- Etc.

```
class C1 {  
    public function method(array $a) {}  
}  
  
class C2 extends C1 {  
    public function method(int $a) {}  
}
```



PHP OOP. *__call()*

- If an object of that class is called with a method that does not exist, the *__call()* is called instead

```
class Dynamic {
    public $message = 'Hello World';
    static protected $methods = array();
    public static function registerMethod($method) {
        self::$methods[] = $method;
    }
    private function __call($method, $args) {
        if (in_array($method, self::$methods))
            return call_user_func_array($method, $args);
    } ...
}

function test($obj) {
    print $obj->message.PHP_EOL;
}

Dynamic::registerMethod('test');
$d = new Dynamic();
$d->test();
```



PHP OOP. *this*

- *this* is not available in a static context

```
class A {  
    function me() {  
        if (isset($this))  
            echo '$this is: ('.get_class($this).')\n';  
        else  
            echo '$this is not defined.\n';  
    }  
}  
  
$a = new A();  
$a->me();           // $this is: (A).  
A::me();           // $this is not defined.
```



PHP OOP. *new*

■ Creating objects:

```
class A {  
    function getNew() {  
        return new static; //late static bindings  
        // => instantiates the subclass - polymorphism  
    }  
}  
  
$a1 = new A();  
$a2 = new $a1;      // $a1 !== $a2  
$a3 = A::getNew();  
$className = 'A';  
$a4 = new $className(); // ⇔ new A()
```



PHP OOP. cloning

- Object cloning

- `__clone()` to implement deep copy

- ```
class MyCloneable
{
 public $object1; // deep copy
 public $object2; // will be cloned as reference

 function __clone()
 {
 // Force a copy of this->object1, otherwise
 // it will point to same object.
 $this->object1 = clone $this->object1;
 }
}
```





# PHP. Other *magic methods*

- **void `__construct` (...)**
  - Child must call explicitly the parent constructor - `parent::__construct()`
- **void `__destruct` ( void )**
  - Child must call explicitly the parent destructor - `parent::__destruct()`
  - Attempting to throw an exception from a destructor could causes a fatal error
- **`serialize()`, `__sleep()`, `unserialize()`, `__wakeup()`**  
generates/recover a storable representation of a value



# PHP 5.3 Garbage Collector

- Memory cleanup for long-running scripts

```
gc_enable(); // Enable Garbage Collector
var_dump(gc_enabled()); // true
var_dump(gc_collect_cycles());
 // # of elements cleaned up
gc_disable(); // Disable Garbage Collector
```

- Cleanup when is no longer a symbol in any scope pointing to the structure



# PHP 5.3 GC Example

---

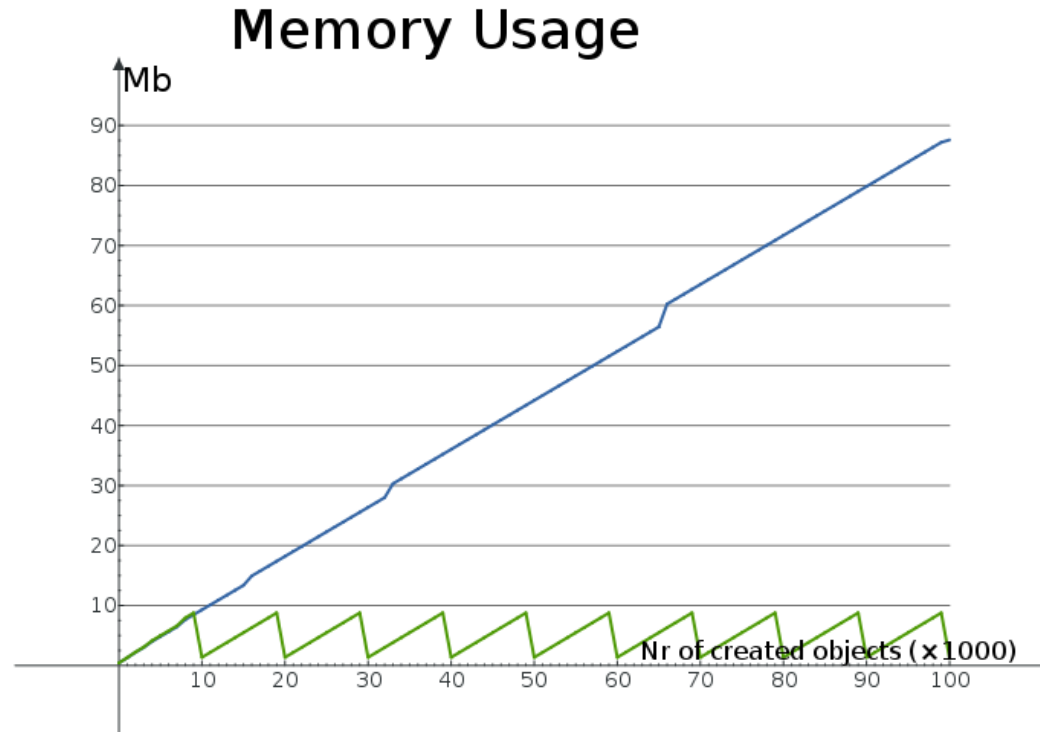
```
<?php
class A
{
 public $var = '3.1415962654';
}

$baseMemory = memory_get_usage();
for ($i = 0; $i <= 100000; $i++)
{
 $a = new A;
 if ($i % 500 === 0)
 {
 echo sprintf('%8d: ', $i), memory_get_usage() -
 $baseMemory, "\n";
 }
}
?>
```



# PHP 5.3 GC Results

■ PHP 5.2  
■ PHP 5.3



\*from [www.php.net](http://www.php.net)



# PHP Documentation

---

- phpDocumentor is the current standard tool for documenting the php code
- Open source - <http://www.phpdoc.org/>
- Similar to Javadoc
- Written in PHP
- Command line and web interface
- Can generate: HTML, PDF, Windows Helpfile CHM, and Docbook XML



# PHP Resources

---

- PHP Tutorial

<http://www.w3schools.com/php/default.asp>

- PHP Examples

<http://www.java2s.com/Code/Php/CatalogPhp.htm>



# Java (Jakarta) - Servlets

- Server-side execution - generates dynamic contents

```
// Import required java libraries
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {

 private String message;

 public void init() throws ServletException
 {
 // Do required initialization
 message = "Hello World";
 }

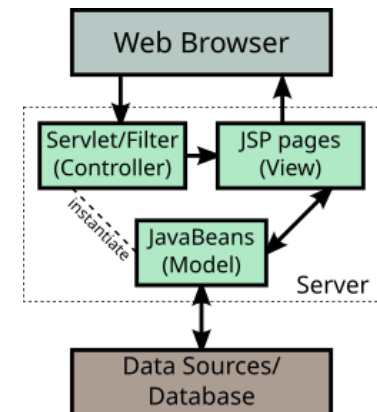
 public void doGet(HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException
 {

 // Set response content type
 response.setContentType("text/html");

 // Actual logic goes here.
 PrintWriter out = response.getWriter();
 out.println("<h1>" + message + "</h1>");
 }

 public void destroy()
 {
 // do nothing.
 }
}
```

Ref. [geeksforgeeks.org](https://www.geeksforgeeks.org)





# Jakarta Servlets

---

- JS - receives a request and generates a response based on that request
- Servlets typically generates HTML content from Java code
- Two technologies
  - Jakarta RESTful Web Services (JAX-RS 2.0) for AJAX and REST services
  - Jakarta XML Web Services (JAX-WS) for SOAP Web Services
- JS runs on a Web Container (e.g., Tomcat) 





# JSP

---

- JSP – Jakarta (Java) Server Pages
  - Allows development of web-based applications that are server and platform independent
  - JSP Technology is included in Jakarta EE Platform (supports JDBC, EJB, JNDI etc.)
  - JSP pages can be used in combination with servlets that handle the business logic
  - JSP vs. Pure Servlets - it is more convenient to write regular HTML than to have plenty of `print()` to generate the HTML





# JSP tags

```
1 <%@page import="java.time.Month"%>
2 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
3 pageEncoding="ISO-8859-1"%>
4 <!DOCTYPE html>
5 <html>
6 <head>
7 <meta charset="ISO-8859-1">
8 <title>Insert title here</title>
9 </head>
10 <body>
11
12 <!-- Expression tag example -->
13 Converting a string to uppercase:
14 <%=new String("Hello World").toUpperCase()%>
15
16 <!-- JSP Scriptlet Example -->
17
18 <%
19 for (Month month : Month.values()) {
20 out.println(month);
21 }
22 %>
23
24 <!-- JSP Declaration Tag Example -->
25
26 <%!int cube(int n) {
27 return n * n * n;
28 }%>
29 <%= "Cube of 3 is:" + cube(3)%>
30
31 </body>
32 </html>
```

**JSP expression tag with syntax** `<%= expression %>`

**JSP scriptlet tag with syntax** `<% statements; %>`

**JSP declaration tag with syntax** `<%! Declaration %>`

Ref. [www.javaguides.net](http://www.javaguides.net)



# Enterprise Jakarta Beans

---

```
import javax.ejb.Stateless;

@Stateless(name="HelloBean")
public class HelloBean
implements HelloBeanRemote, HelloBeanLocal {

private String message = "Hello, ";

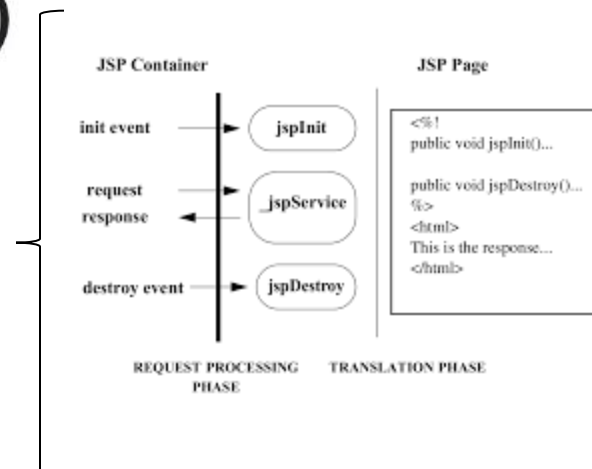
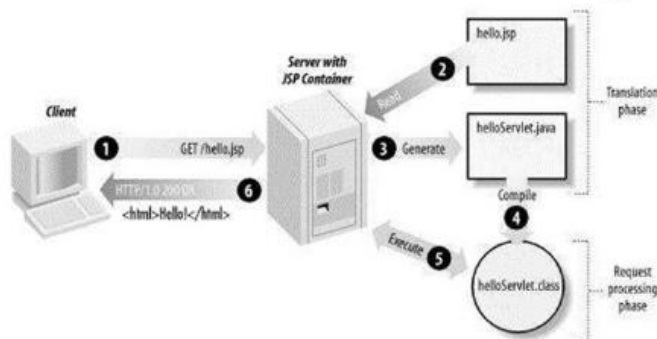
public String sayHello(String name) {

return message + name + ".";
 }
}
```



# JSP Processing

## JSP Processing (in Jakarta Server Pages)





# Microsoft ASP /



- ASP – Active Server Pages from Microsoft
  - Runs natively inside Microsoft Internet Information Server
  - Allows dynamically edit, change, or add any content of a Web page
  - Access a databases from an ASP Web app
  - ASP use VBScript as programming language
  - ASP.NET use C# as programming language
  - ASP supports many different development models as ASP Classic, Web Forms, MVC, Web Pages, ASP.NET API, ASP.NET Core



# ASP.NET tags

## C# Example

```
<!-- Single statement block -->
@{ var myMessage = "Hello World"; }

<!-- Inline expression or variable -->
<p>The value of myMessage is: @myMessage</p>

<!-- Multi-statement block -->
@{
 var greeting = "Welcome to our site!";
 var weekDay = DateTime.Now.DayOfWeek;
 var greetingMessage = greeting + " Today is: " + weekDay;
}
<p>The greeting is: @greetingMessage</p>
```

Ref. w3schools.com



# Node.js (I)



- Node.js

- run javascript (compiled) on the server side as event-driven asynchronous callbacks
- node-js - based on Google V8 VM (same as Chrome)

```
var http = require("http");
function onRequest(request, response) {
 response.writeHead(200, {"Content-Type": "text/plain"});
 response.write("Hello World");
 response.end();
}
http.createServer(onRequest).listen(8888);
```

- Execution of the server: **> node server.js**



## Node.js (II)

---

- Use of JS on both the client and server-side should remove need to “context switch”
  - Client-side JS makes heavy use of the DOM, but no access to files/databases
  - Server-side JS deals mostly with files/databases, but no DOM





# Node.js (III)

- Non-blocking I/O - attaching callbacks to I/O requests

Using the File System module as an example, this is a **synchronous** file read:

```
1 const fs = require('node:fs');
2
3 const data = fs.readFileSync('/file.md'); // blocks here until file is read
```

JavaScript

 Copy to clipboard

And here is an equivalent **asynchronous** example:

```
1 const fs = require('node:fs');
2
3 fs.readFile('/file.md', (err, data) => {
4 if (err) throw err;
5 });
```

JavaScript

 Copy to clipboard

Ref. [nodejs.org](https://nodejs.org)



# Node.js Resources

---

- Node.js homepage

<http://nodejs.org/>

- 15 Best Node.js Tools for Developers – Brainhub

- <https://brainhub.eu/library/node-js-tools-for-developers>

- Useful Node.js Tools, Tutorials And Resources

<http://coding.smashingmagazine.com/2011/09/16/useful-node-js-tools-tutorials-and-resources/>